

Seminar 3 · builds on Lecture 2

Three model mechanisms in system design

Embeddings · attention · sampling and tokenization — six working tasks: build, fix, improve

Three blocks, six tasks

Block 1 · Embeddings

Choosing an embedding model for your corpus · deduplicating support tickets

Block 2 · Attention

Extracting "who pays whom" from contracts · a legal-team assistant and how to measure quality

Block 3 · Sampling and tokenization

Product card generator (description + specs) · tokenization breaks counting and validation

Every case: context → solution and schema → a complication from practice → breakdown with an example



Block 1

Embeddings: comparing and grouping by meaning

Building embedding search · improving support

Which embedding model to use for your corpus

We're building semantic search over an internal knowledge base: an employee asks in plain words, the system finds the document by meaning. The first decision: which model encodes text into vectors.

Corpus

in-house, Russian-language, with domain-specific terms

Output

return relevant documents for a query

Latency

results within ~300 milliseconds

Switching models is costly

change model → reindex the whole corpus

No labeled data

no table of "for query X, Y and Z are relevant" exists

Out of scope

the selection criterion, model list, dimensionality — these are decisions, not givens

What criterion should we use to choose a model — and how do we test candidates on OUR OWN data?

How to choose: eval first, everything else second

Search quality is a property of the pair "model + your corpus + your queries," not of the model alone. It can only be measured on your own data: how well the model's vectors bring YOUR queries close to YOUR relevant documents.

How it works

- Pick 3-4 candidates from different classes via MTEB/ruMTEB (the embedding-model leaderboard; ru- is the Russian version) — as a shortlist, not as an answer
- Build a mini-eval: 50-100 "query → relevant document" pairs from your corpus, plus traps
- Run each candidate on the SAME set → recall@k and MRR under identical conditions
- Weigh quality against vector dimensionality: 384 vs 1024 → storage and search latency
- Choose along the "quality on my data / cost" axis, and pin the model version in the config

Candidates (from leaderboard)

Mini-eval (your pairs)

recall@k · MRR

Choice: quality / cost

recall@k is the share of queries where the right document lands in the top-k. MRR (mean reciprocal rank) shows how high it ranks. Eval set size is a function of the task (query diversity, cost of error), not a round "200" — even 30 honest pairs beat choosing blind.

Three axes of choice — each checked by the same eval

The axes don't reduce to one number: quality on your data, vector dimensionality, cost, and latency. Different model classes offer different trade-offs on each axis.

Multilingual / monolingual

Multilingual — broad coverage;
Russian-centric — richer Russian,
narrower coverage.

- Multilingual: e5, bge-m3 — average quality across dozens of languages
- Russian-centric: RoSBERTa, GigaEmbeddings — candidates for a Russian domain
- Tested by: recall@k on a Russian eval

Domain-specific / general

Domain-specific/fine-tuned for legal/med/fintech vs. a solid general model out of the box.

- Domain-specific may score higher on its own domain
- General is sometimes good enough and cheaper
- Tested by: eval; "domain-specific = better" is a hypothesis

Dimensionality

Compact 384-768 vs. large 1536-3072 (MTEB flag, bigger vector).

- Large: higher average score, costlier storage and search
- Compact: cheaper, closer to the 300ms budget
- Tested by: equal recall → pick the smaller one

Don't compare models by description — compare them by numbers on your own corpus. Larger dimensionality linearly increases vector-DB size and search time across the WHOLE index: if a 768-dim model gets the same recall@10, there's no reason to pay for 1536. Vector truncation (Matryoshka/MRL) is checked with the same measurement.

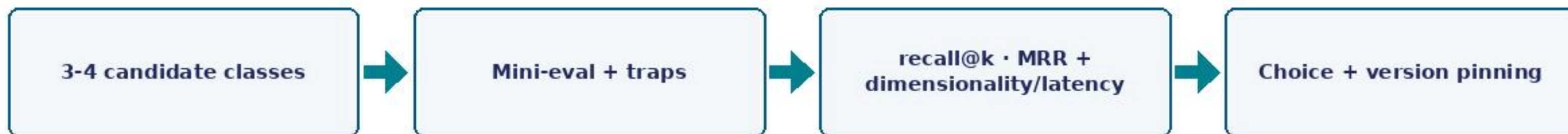
The benchmark leader slips exactly on the Russian domain

In a documented production system for searching legal contracts, the top-3 MTEB leaderboard models placed 5th, 7th, and 2nd on the in-domain eval set: the model ranked first on the general benchmark came in fifth on the real corpus — beaten by less "decorated" ones.

The benchmark's average score is averaged across dozens of languages and domains. "Best on MTEB" and "best on your data" are different models — quality isn't a single axis.

Selection pipeline: leaderboard → eval → metrics → choice

The leaderboard shortlists candidates; from there, measurement on your own data makes the decision.



- MTEB/ruMTEB leaderboard is only a source of candidates, not the answer
- Every candidate is run on the SAME set under identical conditions
- Choice is made on the "quality on my data / cost" axis; the winner's version is pinned — switching means reindexing

An hour of eval versus weeks of living with a bad, expensively-reindexed index.

Breakdown · components and commentary

Key components

Candidate list

3-4 models from different classes via MTEB/ruMTEB, used as a shortlist; BM25 can be added as a non-AI baseline.

Mini-eval set

A table of "query | document id | type (regular/trap)," 50-100 rows. The sole arbiter of quality.

Encoder

The text-to-vector model; query/document modes and metric/normalization matching the model's training both matter.

Nearest-neighbor index

pgvector / Qdrant / FAISS — stores the corpus vectors, quickly returns the top-k.

Pinned config

A locked model version + metric + dimensionality — protection against a silent update.

Key commentary on the topic

- The vector reflects the statistics of what the model saw during training → "search quality" is inseparable from the pair "model + your data."
- similarity $\neq$ relevance: high similarity for pairs like "turn on/turn off" is a limit of the method itself; a reranker/hybrid removes it, not a model swap.
- Pitfalls: metric/normalization mismatched with training; encoding query/document the same way; choosing by leaderboard without an eval; "I'll just take a bigger one to be safe."

Breakdown · how to build it

Choosing an embedding model = a mini-eval on your own pairs plus a deliberate dimensionality trade-off; the leaderboard only supplies candidates, and the version gets pinned in the config.

How to build it

- Gather 50-100 "query → relevant document" pairs from your corpus, plus "on/off" traps — that's your eval
- Run candidates (multilingual, Russian-centric, large, BM25) on the SAME set → recall@k and MRR
- Weigh quality against dimensionality: if 768 gives the same recall as 1536, pick 768
- Check that the metric (cosine/dot) and normalization match how the model was trained; pin the version

V An hour of eval versus weeks with a bad index

Run the final check on a sample of your own documents and queries: an hour of work versus weeks with a bad index. The MTEB leader $\neq$ the leader on your domain.

Collapsing duplicates in a stream of support tickets

A payment service's support team receives a stream of tickets; the same problem arrives over and over, phrased dozens of different ways. We want to merge similar ones automatically.

Stream

on the order of hundreds of tickets a day, continuous

Duplicates

one problem, different wording, often with no shared words

Structure available

card type, error code, service id

Almost no labels

no ready "duplicate / not duplicate" set for our stream exists

Asymmetric cost of error

merging different tickets hides and loses an incident

Decision on arrival

at intake, not batched once a day — otherwise an agent handles the duplicate by hand

How do we find and merge duplicates — and where do we put a safeguard against false merges?

How the dedup works — and why a structural gate

Embedding-based dedup: we encode ticket text into a vector; cosine-close ones become candidates for the same cluster past a threshold τ . A structural gate is a set of hard fields that must match for a merge to be allowed.

How it works

- Vectorize the ticket text → similar wording yields similar vectors, even with no shared words
- Cosine threshold τ : pairs with similarity $> \tau$ are merge candidates ("does it look similar?")
- Structural gate: merge ONLY if the keys match — card type, error code, service ("is it the same thing?")
- Keys don't match despite high similarity → don't merge, flag for manual review

Ticket → vector

Similarity $> \tau$?

Structural keys match?

Merge / keep separate

The threshold τ isn't a constant from an example — it's a separate tuning step: it's set from labeled pairs via a precision-recall curve. The size of the labeled set is a function of the task (stream frequency, cost of error, wording diversity), not a round "200."

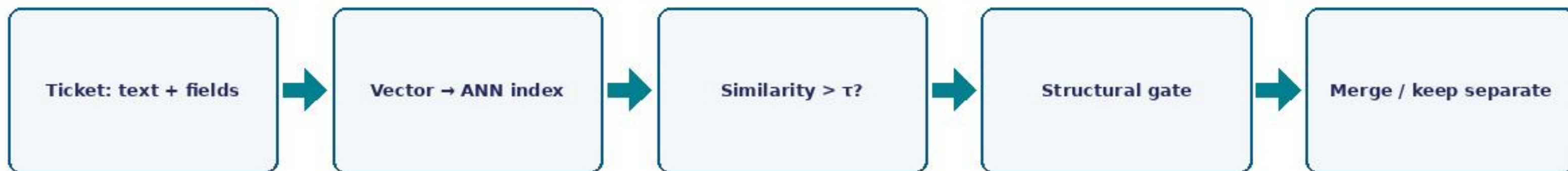
"Visa" and "Mastercard": close by vector, not the same thing

"Payment fails with a Visa card" and "payment fails with a Mastercard card" are near-identical by vector (cosine 0.95+): one word out of six differs. But these are different incidents — different acquiring, different gateways, different causes. A plain similarity threshold merges them and hides one incident inside the other.

The embedding encodes the TOPIC and systematically loses the distinguishing detail. Raising τ doesn't help: any τ that separates Visa/Mastercard will also throw out a mass of real duplicates.

Hybrid "similarity + structural keys"

Text decides "does it look similar," deterministic fields decide "is it the same thing." ANN is approximate nearest-neighbor search (quickly finds similar vectors).



- Similarity gathers candidates cheaply and broadly (incrementally, via an ANN nearest-neighbor index)
- A structural gate on required fields filters out false merges precisely and safely
- High similarity + mismatched keys → don't merge, send to manual review, not silently into a new cluster

Neither mechanism works alone: similarity merges near-duplicates, and keys without similarity miss duplicates.

Breakdown · components and commentary

Key components

Normalizer / field extractor

A ticket from any channel → clean text + structural fields (regex/parser).
The gate's reliability depends on it.

Embedding model

Text → vector (chosen in Case 1.1); the metric and normalization must match its training.

ANN index

FAISS / Qdrant / pgvector — incremental neighbors, no quadratic recompute.

Similarity threshold τ

The only "soft" parameter; set from a precision-recall curve for a fixed target precision.

Structural gate

A deterministic "all keys match" — carries the distinguishing detail that the vector loses.

Manual review queue

Disputed near-duplicates go to a human — a safeguard against losing an incident and a source of new traps.

Key commentary on the topic

- The embedding encodes "what a text is about" and loses the distinguishing detail when the lexical difference is small — cosine gives candidates, not identity.
- The main pitfall is trying to separate near-duplicates by raising τ : any τ high enough to do that will throw out real duplicates. Structure is what catches the distinguishing detail.
- Asymmetric cost of error → priority goes to precision, not balance; missed recall is recovered via the hybrid and manual review.

Breakdown · how to build it

Dedup = vector similarity plus required structural keys: text decides "does it look similar," keys decide "is it the same thing." Disputed cases go to manual review, not silently through.

How to build it

- Extract structural fields (card type, error code, service) — regex/parser from machine-readable sources
- Set τ as a separate step: label duplicate/not-duplicate pairs (with plenty of traps), build precision-recall, pick τ for the target precision
- Merge candidate = similarity $> \tau$ AND all required keys match
- Disputed cases (high similarity, mismatched keys) aren't merged — they go to manual review; at release we measure the false-merge rate

Confident false merges (near-duplicates)

"Pure text similarity produces confident false merges": semantic similarity doesn't guarantee identity — the distinguishing detail is caught by the structural gate, not the threshold.



Block 2

Attention: what the model understands in text

Building extraction · improving an assistant

Extract "who pays whom, how much, and when"

The legal department has accumulated hundreds of contracts. As long as payment obligations live only in text, you can't use them as data. The task: turn free text into structured payment records.

Input

contracts in different formats: docx, native PDF, scans

Extracting

payer, payee, amount, due date, condition

Money fields are critical

an error in the amount or direction = a wrong posting

All by hand -- no

hundreds of contracts, thousands of fields you can't check manually

Auditability

for every value, see exactly where in the text it came from

Out of scope

sufficiency criteria; where the real difficulty lies is the subject of the breakdown

What should the pipeline look like -- and where should you place verification for critical fields?

Value + span evidence + confidence gate

Attention reliably resolves "who to whom" as long as the text carries an unambiguous signal, and errs systematically when there is no unambiguous answer. So: bulk extraction goes to the model, critical fields go under non-model verification.

How it works

- For EVERY field, the model returns a triple: value + evidence_span (a verbatim quote) + a confidence flag
- the span is checked as a substring of the source: not found → a signal of fabrication, the field is disputed
- Structured output fixes the form: the model must return the answer per a JSON schema; strict = it won't return a field outside the schema (the response_format parameter) -- it guarantees form, NOT truth
- Deterministic cross-checks in code: the amount in words matches the amount in figures, dates are valid, the parties are unambiguous
- Gate: no span / low confidence / failed check / a pronoun as the evidence → to a human

Contract

LLM extraction + span

Cross-checks (code)

Confidence gate

Human review on disputed items

Take confidence from objective signals (presence of a span, cross-checks, log-probabilities), not from asking the model "are you sure?" -- verbal self-assessment measures how well the model was trained to sound confident, not its actual reliability.

"...if it provides the guarantee" -- who is "it"?

"The Customer pays the Contractor an advance if it provides a guarantee." Who is required to provide the guarantee? Grammatically, both nouns are neutral -- agreement doesn't narrow it down. WinoBias (a coreference benchmark, 3160 sentences): changing a single word flips the referent, and systems link pronouns to pro-stereotypical entities 21.1 F1 points more accurately (an accuracy metric, 0-100).

The attention mechanism relies on statistical correlation between words, not on cause and effect. On ambiguous coreference, it errs systematically, in a predictable direction -- and in a confident tone.

LLM extraction with verification of critical fields

The model is a fast first pass; verification goes where the cost of error is high.



- Garbage input (bad OCR) doesn't get through -- a text-quality threshold sits before extraction
- Deterministic checks catch mismatched sums and invalid dates without the model
- No span / low confidence / pronoun as evidence → to a human; money never reaches production past the gate

Automate the bulk, insure the critical -- not "all or nothing."

Breakdown · components and commentary

Key components

Input normalizer

OCR (Tesseract/Yandex Vision/ABBYY) + pdfplumber/PyMuPDF + a text-quality threshold. The model gets text, not noise.

LLM extractor

A top-tier model with structured output (json_schema, strict); the stable part of the prompt goes first (prefix caching).

Cross-check layer

num2words / a spelled-out-number parser, dateutil/workalendar, currency and party reference tables, pydantic. Errors caught without the model.

Confidence gate

span found $\wedge$ confidence $\geq$ threshold $\wedge$ checks passed $\wedge$ evidence isn't a pronoun. Splits auto vs. human.

Human review queue

A table of disputed fields with the quote and the contract -- a focused review of 5-15% of fields.

Winograd-style eval set

Disputed cases with ground-truth answers -- measures coreference on every release, catches regressions.

Key commentary on the topic

— Attention relies on the statistical correlation of words, not on causation: on unambiguous cases it's excellent, on ambiguous ones it systematically picks the frequent option.

— A span, not "are you sure": a confident tone doesn't correlate with whether an unambiguous answer exists; the quote is either in the text or it isn't.

— The schema guarantees the form of the JSON, not the truth of the value -- truth comes from the cross-checks and the gate, not from a strict schema.

Breakdown · how to build it

Extraction is the model's job; resolving ambiguous coreference in money fields goes under verification and rules. The schema gives you form, code and the gate give you truth.

How to build it

- The prompt returns, for every field: value + span evidence from the text + a confidence flag
- Deterministic cross-checks: the amount in words matches the amount in figures, dates are valid, parties resolve to a dictionary built from the header
- Gate: a field with no unambiguous span, low confidence, or a pronoun as evidence → goes to manual review
- Keep an eval set of ambiguous contracts (Winograd-style), measure the share of correct referents and the share of escalations

X Hallucinations even with RAG (legal/finance domain)

Stanford, 2024: source-grounded tools still hallucinated -- Lexis+ AI 17%, Westlaw 33%, plain GPT-4 43%. Money fields without verification are unacceptable.

"It answers inaccurately -- make it better"

A chat assistant for the legal department has been running on a general-purpose model for a year. The client shows up with a vague "make it answer better": the answers sound confident, but the facts and the references to regulations are regularly inaccurate.

What exists

a working assistant on a general-purpose model that doesn't use the regulations database

Complaint

confident tone, but facts and regulation citations are inaccurate

No metric

"quality" exists only as a feeling -- no number, no baseline

Data available

current regulations and rules in machine-readable form

Logs available

thousands of real questions from lawyers over the past year

Cost of error

a wrong citation to a regulation → legal risk

How do you raise quality -- and how do you prove you raised it, rather than just made it "sound more authoritative"?

Metric and baseline first, changes second

You can't improve what you don't measure. A generative system is deceptive: its output always looks like an answer, and to the naked eye a correct answer and a confident fabrication are indistinguishable. You need a reproducible measuring instrument -- a golden set.

How it works

- Build a golden set: 50-100 real questions from the logs with reference answers and correct citations
- Three metrics: factual accuracy · citation correctness · share of answers with a verifiable source
- Score automatically: meaning -- LLM-as-judge (a stronger model scores the answer against a rubric, calibrated), citation -- exact string match (code, not a judge)
- Measure the current assistant's baseline on the set -- that's your reference point
- Change one thing at a time and measure on the same set; a change is accepted only if the metric improves

Question

Assistant's answer

Check against reference / regulation

3 metrics

A golden set is just a table: "question | reference answer | correct citation." Its size is a function of the task (traffic diversity, cost of error, number of topics), not a round "200"; the sufficiency criterion is that the set has stopped surfacing new types of errors.

"You are an experienced lawyer" doesn't move the metric

A common first attempt is adding a role to the prompt: "You are an experienced lawyer with 20 years in practice." Zheng et al., 2024: 162 persona roles across 2410 factual questions, four model families -- adding a role does NOT raise accuracy; the persona effect is essentially random. On the eval set, answers become more confident in form, but not more correct in substance.

The role uses the same attention mechanism that resolved coreference: role tokens shift the choice toward style, but they don't add knowledge. Form and knowledge are different axes. Accuracy comes from grounding, not from a "title" in the prompt.

Breakdown · components and commentary

Key components

Golden set

A table of "question | reference answer | correct citation" from real logs. The only anchor that tells improvement from regression.

Set builder

Clustering the logs by embeddings + selection for even topic coverage. Measures real traffic.

Eval runner

Ragas / DeepEval / Promptfoo -- runs a version against the set, collects the three metrics.

LLM judge

A stronger model, scores meaning against a rubric, T=0, calibrated against a manual sample.

Deterministic check

Code: normalizing and comparing citations, checking that the quote exists. No hallucinations, no drift.

RAG grounding on regulations

Retrieving fragments from the database + answering from them with a mandatory citation. The real lever on accuracy (per the metric).

Key commentary on the topic

- "Raise quality" is first a measurement problem, then a change problem: without a number, any edit is blind, and a confident tone and a fabrication look the same to the eye.
- Form and knowledge are different axes: a role moves form (the attention weight of the role's tokens), knowledge comes from data in context (grounding).
- grounding is a lever, not an off switch: even RAG-based tools give 17-33% errors; a change is accepted based on the metric, the judge is calibrated, citations are checked in code.

Breakdown · how to build it

Golden set and baseline first; what raises quality is grounding on current regulations, verified by the metric -- not the wording of a role. Change one thing at a time.

How to build it

- Build a golden set (question/reference answer/citation) from real logs, measure the current assistant's baseline
- Test the "role" hypothesis: with the role and without it -- the metric is the same (that's a success for the eval, not a failure)
- Add RAG on the regulations database with a mandatory citation; accuracy and grounding rate rise -- accept it
- Automated check: accuracy -- LLM judge (calibrated), citation -- exact string match; accept a change only if the metric improves

Role instead of measurable quality

Zheng et al., 2024: 162 persona roles, 2410 questions -- role-in-prompt improves form, not factual accuracy. The rise in accuracy comes from grounding, and you see it on the eval, not by eye.



Block 3

Sampling and tokenization: how the model generates and sees text

Picking a generation mode · running into the limits of counting

Product cards: description and structural parameters

A marketplace creates product cards by the thousands daily. Each card has two fundamentally different field groups: a marketing description (free text) and structural attributes (machine-readable fields).

Volume

thousands of cards a day — writing them by hand isn't realistic

Description

natural, coherent text, one consistent brand voice, not copy-paste

Attributes

dimensions, weight, category from a reference list — a strictly defined form

Regulated fields

ingredients, shelf life, warranty — the source of truth is the GOST standard/spec sheet

Cost of error differs

marketing — cheap and reversible;
regulated fields — liability to the regulator

Which mode to set — and is it the same for every field?

One prompt, three temperatures: an axis of spread, not of "flair"

The same factual question, three temperatures, repeated runs at each. The axis is "predictable/repeatable ↔ random/varied and risky".

T = 0 (repeatable)

Repeated runs — identical answers.
The choice is almost deterministic.

- Predictable and repeatable
- Classification, extraction, attributes
- No spread, by construction

T = 0.7 (working spread)

Phrasing drifts between runs, but the facts and coherence hold.

- Variety of form without losing substance
- Marketing, explanations
- The working choice for descriptions

T = 1.3+ (risk)

The answer falls apart: coherence breaks down, factual errors and stray insertions creep in.

- Idea drafts, brainstorming
- Not for facts
- Risk of incoherence and fabrication

Mini-experiment: 20-30 cards from different categories, one prompt at 0.4 / 0.7 / 1.0 / 1.3, 3 runs each, scored on "variety of phrasing without factual errors and in the brand tone". The working zone is usually 0.6-0.8; the chosen T is locked into the config, not the provider's default.

Temperature sets spread; a schema sets form, not truth

Temperature — one knob: how random the choice of the next word is. Low → the choice is predictable and repeatable (the model almost always takes the most probable option); high → the choice is more random and varied, but riskier. It doesn't change the order of "what's more probable", or the facts themselves.

How it works

- Low T → almost always the most probable option; predictable and repeatable (at T=0 — argmax)
- High T → rare options get a chance; more varied, but incoherence and fabrication creep in
- T ≠ truncation: top-k / top-p CUT OFF unlikely options (by count / by mass), T only changes the randomness of the choice
- structured output (json_schema, strict) — the model must return an answer matching a JSON schema; strict = it won't return a field outside the schema (the response_format parameter)
- The schema guarantees FORM (numbers as numbers, a category from the reference list), but NOT the truth of the value

Next-word choice

Temperature: randomness

top-k/top-p: cut the rare

Schema: form, not truth

The correct temperature axis is "predictable/repeatable ↔ random/varied and risky", NOT "boring ↔ vivid". High T doesn't make the style more flamboyant — it makes word choice more random. Brand tone is set by the prompt and examples, not by temperature.

Regulated fields: no temperature saves you

Ingredients, contraindications, warranty terms, "complies with GOST..." — the same cards, but variability isn't acceptable here, the text must exactly match the source. Air Canada (2024): a chatbot "made up" a refund policy that didn't exist, and a tribunal held the company liable — the company is responsible for a generated factual statement.

Temperature controls spread, not accuracy. Even $T=0$ makes the output repeatable, not true — the model can consistently repeat something wrong. Regulated fields don't need the "right T" — they need substitution from a verifiable source.

A three-branch pipeline: two runs + substitution

Different groups of fields — different generation modes; regulated fields bypass the model.



- Run A — free text, temperature tuned on a batch, input is known facts only
- Run B — structured output (json_schema, strict) + T=0 + an evidence field for every value
- Regulated fields (ingredients, shelf life, warranty) — substituted from the spec sheet/GOST standard WITHOUT generation; anything suspicious goes to a human

One call for everything is a false saving of one line of code, paid for with a worse result in both groups of fields.

Breakdown · components and commentary

Key components

Field classifier

A deterministic layer (rules/config): generate freely, generate structurally, or take from the source. Implements "different fields — different modes".

Run A (description)

Free text: a prompt with brand tone + few-shot examples, $T \approx 0.6-0.8$ from the config, input is known facts only.

Run B (attributes)

structured output (response_format/json_schema/strict), $T=0$, a schema with an evidence field. Valid form.

Value validator

Code after B: ranges, units, evidence present in the input, category from the reference list. The truth the schema doesn't provide.

Regulation substituter

A template + source (spec sheet, GOST standard, warranty reference). Verbatim substitution, no generation.

Config

Model version, temperatures per field type, schema version — a single point for deliberate decisions.

Key commentary on the topic

- Temperature = the spread axis, not style and not accuracy: high T increases the randomness of token choice, and on facts it leads to fabrication.
- A schema guarantees form, not the truth of a value: `weight_kg=5.0` can be fabricated → value validation and an evidence field are mandatory.
- $T=0 \neq$ truth (and not full determinism in the cloud either): no T saves regulated fields — their source of truth is the spec sheet/GOST standard, not the model.

Breakdown · how to build it

Different fields — different modes: $T \approx 0.7$ for marketing, structured output + $T=0$ for attributes, substitution from the source for regulated fields. The schema gives form; code and the source give truth.

How to build it

- Split the card's fields with a classifier: marketing (generate), structural (generate structurally), regulated (substitute)
- Marketing: $T \approx 0.6-0.8$, tuned on a batch of 20-30 cards; brand voice via the prompt and few-shot examples, not temperature
- Attributes: structured output (json_schema, strict), $T=0$, an evidence field; then value validation by code
- Regulated: a template with substitution from a verified source, no free generation at any T

Structured output as a reliable form

OpenAI: structured outputs in strict mode give 100% schema compliance versus ~86% for function calling — the model physically cannot emit a token outside the schema.

Character-level features: password, card, limit, count

A web service with registration and payments. A set of small features working at the level of individual characters. At the planning meeting: "We have a model — let's just ask it, it's smarter than regexes."

Feature 1 · password

length ≥ 12 , character classes, no 3+ identical in a row + a list of violations

Feature 2 · card

last 4 digits visible, the rest asterisks (a PCI DSS requirement)

Feature 3 · truncation

exactly to a character limit (SMS segment, DB field), without breaking a word

Feature 4 · counting

character count, length, an SKU of exactly 8 characters

Precision is mandatory

code runs on every request, no human in the loop; one wrong character = a leak or a rejected form

What should implement each feature, and why — a model, code, or a combination?

Why the model structurally can't count characters

A token is an identifier from the model's vocabulary. Text is cut into tokens (frequent subsequences) and turned into numbers; from then on the model works ONLY with numbers, not letters. Token boundaries don't line up with letter boundaries.

How it works

- "strawberry" is 3 tokens: [st][raw][berry], not 10 letters; "klubnika" (Russian for "strawberry") is also 3 tokens
- Inside the token [raw] there's NO separate representation of the letter 'r' at position 1 — just averaged statistics of contexts
- The vocabulary is built once, before training, by the BPE algorithm (Byte-Pair Encoding — how text gets cut into tokens), and doesn't change afterward — a trade-off made for the sake of attention's economics
- A character-level representation would make the input 3-5x longer, and the cost of attention grows quadratically
- So character counting, length, and character-level slicing are unreliable — and this is NOT something that "gets fixed in the next version"

String "strawberry"

BPE cuts it into tokens

[st][raw][berry] = 3 numbers

No letters as units

Jagged intelligence: the same model solves an olympiad problem and fails to count the letters in a 9-letter word. Success on an impressive task is NOT evidence of reliability on a "simple" one. "It wrote SQL — surely it can count characters" is a mistaken extrapolation.

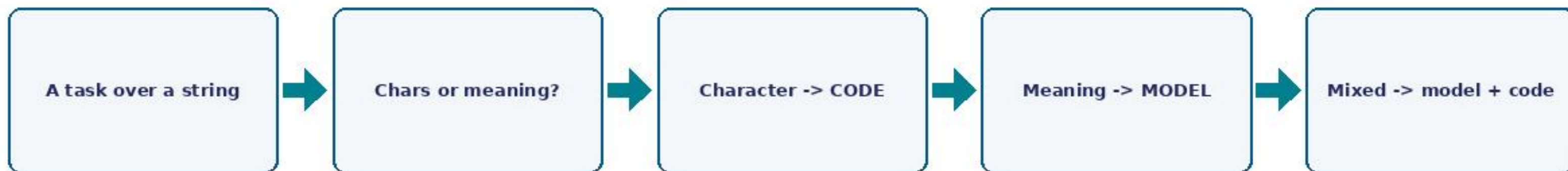
Patch != skill: strawberry fixed, cranberry breaks

A race of point patches: GPT-5.2 answered "two r's" for strawberry; GPT-5.5 fixes strawberry, but for "how many r's in cranberry" answers "two" (the correct answer is three); GPT-5.6 fixes cranberry. Every viral question eventually gets its own fix, but the task class never gets solved.

Passing a viral specific example $\neq$ having a skill. The model passes KNOWN examples (fine-tuned in, or via a heuristic), but your SKU or password is unknown to it. Swap in a rare, non-viral word and the failure comes back.

Deterministic to code, semantic to the model

Split the task by the nature of the result: an exact character-level result, or understanding of meaning.



- An exact result at the level of characters/length/format/numbers → code: len, regex, slices, count, formatters
- Understanding meaning (is this spam? what's the tone? paraphrase this?) → the model, it has no competition here
- Mixed (shorten to under 160 characters while keeping the point): the model does meaning, code does the exact length (truncate on top)

Code: microseconds, free, 100% reproducible, testable. A model's API: tens to hundreds of ms, paying for tokens, a probabilistic result.

Breakdown · components and commentary

Key components

Task-nature classifier

Not code, but an engineer's decision/checklist: "exact, character-level result → code; meaning → model". Made before implementation.

Character code

len (length), regex (classes, format, positions), slices (s[:n], s[-4:]), count, formatters. Microseconds, testable.

Model (LLM)

Meaning only: classification, extracting the gist, paraphrasing, a clear error message. Never exact counting/length/masking.

Model-output wrapper

For mixed tasks: truncate()/validate() on top of the model's answer guarantees the exact form.

Boundary unit tests

Threshold ± 1 , empty string, whitespace, emoji/non-ASCII, a long string. Catch off-by-one errors.

Tokenization visualizer

tiktokenizer.vercel.app / tiktoken — a tool for understanding and demonstration, not for runtime use.

Key commentary on the topic

- Inside a token there's no reliable representation of "which letter is at which position" → counting, length, slicing, format validation are structurally not the model's job.
- The 3.1/3.2 pair is one nature of the model seen from two sides: know where the mechanism works for you (generation) and where it works against you (counting).
- Traps: extrapolating from hard to "simple" (jagged intelligence); "it answered right on my test" (check a rare case); sensitive data sent to an API; no boundary tests.

Breakdown · how to build it

Code handles character-level and deterministic work; the model is for meaning, not counting. If you can't avoid the model, it handles meaning while wrapper code guarantees the exact form.

How to build it

- Password — len + regex over character classes; card — a string slice (d[-4:] + asterisks), locally, not a byte goes to an API
- Truncation to a limit — a slice + len; counting and format — count and re.fullmatch; all deterministic and in microseconds
- A mixed task — the model handles meaning, code guarantees form: truncate(model_output, 160) on top
- Set up boundary unit tests: threshold ± 1 , empty string, whitespace, emoji/non-ASCII, a long string



Code where code belongs, the model where the model belongs

Production form-validation, masking, and truncation systems do NOT use an LLM for the character-level part — that's regex, len, slices, covered by tests. The LLM only comes in for meaning.

What to take away

- Embeddings: which model to pick is decided by a mini-eval on your own data (recall@k/MRR), not by a benchmark ranking; "similar" $\neq$ "the same" $\rightarrow$ use structural keys.
- Attention: correlation, not causation $\rightarrow$ the model extracts and justifies a span, rules and a human verify anything critical; grounding moves quality, not role-play.
- Sampling and tokenization: temperature sets the spread (not style, not truth), a schema sets the form (not truth); counting characters isn't the model's niche — code does it.

One principle across all three: a mechanism isn't a "model feature" — it's a boundary the engineer knows from both sides, using its strength and hedging its weakness. Next — Lecture 3: how these mechanisms combine into architectures (RAG, agents).